Part 2: Controlled online SSH dictionary attack

Estimated time: 25 minutes. Environment: Docker Compose course network.

Ethical and authorized use: this exercise may target only the fixed lab01-ssh-target service supplied with the course. Do not modify the client to accept an IP address, public hostname, different username, arbitrary port, or external wordlist. Do not apply this procedure to any other SSH service.

Part 2 demonstrates an online dictionary attack. Every candidate causes an authentication request to a deliberately vulnerable SSH server in an isolated Docker network. Unlike offline hash guessing, the server can observe, log, delay, rate-limit, or block these requests. Part 3 performs the corresponding offline hash-file experiments.

Learning objectives

After completing this part, you should be able to:

- distinguish an online login attempt from an offline verifier check;
- use a fixed and bounded instructor-controlled target safely;
- implement one password-only SSH authentication attempt;
- ensure network resources are closed after success or failure;
- observe why server-side monitoring applies to online attacks; and
- explain why the target and input restrictions are security boundaries.

Controlled environment

The Compose profile creates two services on the internal lab01-ssh-internal network:

```
course container  --->  labstudent@lab01-ssh-target:22
```

The SSH service has no host ports: mapping, uses a fictional non-root account, and receives candidates sequentially from a course wordlist containing at most 20 entries. The starter intentionally does not provide target-selection or concurrency options.

Tasks

1. Build and start the instructor-provided SSH target.
2. Enter the course container on the same Compose network.
3. Inspect the fixed constants and bounded wordlist.
4. Implement only try_candidate() in ssh_dictionary_attack.py.
5. Run the public safety checks.

6. Execute the bounded exercise and record status, attempt count, and time.
7. Inspect the target's local logs and explain what the server can observe.
8. Exit and remove the temporary target container.

Do not print or include the matching candidate in your report.

Step 1: build and start the isolated target

Run these commands from the repository root in a host terminal, not from inside the course container:

```
docker compose -f docker/compose.yml --profile lab01-ssh build
docker compose -f docker/compose.yml --profile lab01-ssh up -d lab01-ssh-target
docker compose -f docker/compose.yml --profile lab01-ssh ps
```

Wait until the target is reported as healthy. Because the service uses only expose: 22 and has no published port, host and external clients cannot connect directly.

Step 2: enter the course container

```
docker compose -f docker/compose.yml --profile lab01-ssh run --rm course bash
cd /workspace/labs/lab01/part2_dictionary_attack
```

Python files and usage examples

‘ssh_dictionary_attack.py‘

The destination and maximum input size are fixed in the supplied code. Implement try_candidate() as follows:

1. Create paramiko.SSHClient().
2. Use AutoAddPolicy only because this target is an isolated, disposable classroom container.
3. Call connect() using the fixed host, port, username, and supplied candidate.
4. Disable SSH agent and local-key lookup so only that candidate is tested.
5. Return False for paramiko.AuthenticationException.
6. Return True after successful authentication.
7. Close the client in a finally block.

Run the bounded exercise only after the checks below pass:

```
python3 ssh_dictionary_attack.py ../data/ssh-lab-wordlist.txt
```

Expected output shape:

```
Target: labstudent@lab01-ssh-target:22
Attempts: N
Elapsed: N.NNN seconds
Result: candidate found
```

The program deliberately reports success without printing the candidate.

‘test_ssh_dictionary_attack.py‘

```
python3 -m unittest test_ssh_dictionary_attack.py -v
```

Expected result: two tests pass. They confirm that the target is the fixed Compose service and the supplied wordlist remains within the 20-candidate limit. They do not test a real network or reveal the solution.

Step 3: observe and remove the target

In a second host terminal, inspect only this container’s recent logs:

```
docker compose -f docker/compose.yml --profile lab01-ssh \
  logs --tail 50 lab01-ssh-target
```

Compare the failed-login records with Part 3’s offline CSV attacks, which produce no server-side signal. Exit the course shell and remove the target:

```
exit
docker compose -f docker/compose.yml --profile lab01-ssh \
  rm --stop --force lab01-ssh-target
```

Online versus offline comparison

Property	Part 2 SSH attack	Part 3 hash-file attacks
Guess reaches a service	Yes	No
Service can log attempts	Yes	No

Network protocol overhead	Yes	No
Server-side rate limiting	Possible	Not applicable
Lab target	Internal Compose service	Local synthetic CSV

Completion criteria

You have completed Part 2 when the target remains isolated, both safety tests pass, the bounded script reports a successful status without printing the candidate, and the target container is removed. Include only the attempt count, elapsed time, redacted log observations, and online/offline explanation in your report.

Checkpoint questions

1. What makes this activity online rather than offline?
2. Which evidence shows that the SSH server can observe the guesses?
3. Why are candidates attempted sequentially with a delay?
4. Why are hostname, port, username, and wordlist restrictions part of the exercise’s safety boundary?
5. Why is AutoAddPolicy acceptable only for this disposable classroom target and inappropriate as a general SSH-client default?
6. Which defensive controls could a production SSH service apply to these requests?